My Project

Generated by Doxygen 1.18.0

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 Element Class Reference . . . 5

3.1.1 Detailed Description . . . 5

3.2 Fabryka Class Reference . . . 5

3.2.1 Detailed Description . . . 6

3.3 Iterator Class Reference . . . 6

3.3.1 Constructor & Destructor Documentation . . . 6

3.3.1.1 Iterator() . . . 6

3.3.2 Member Function Documentation . . . 7

3.3.2.1 nastepny() . . . 7

3.3.2.2 poprzedni() . . . 7

3.3.2.3 ustaw() . . . 7

3.4 ListaDwukierunkowa Class Reference . . . 8

3.4.1 Member Function Documentation . . . 8

3.4.1.1 dodajNaKoniec() . . . 8

3.4.1.2 dodajNaPoczatek() . . . 9

3.4.1.3 dodajPodIndeks() . . . 9

3.4.1.4 usunPodIndeks() . . . 9

4 File Documentation 11

4.1 Element.cpp . . . 11

4.2 Fabryka.cpp . . . 11

4.3 Iterator.cpp . . . 11

4.4 ListaDwukierunkowa.cpp . . . 12

Index 15

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Element
: Tworzy nowy Element listy 5

Fabryka
: Tworzy nowy Element za pomoc[1] operatora new 5

Iterator 6

ListaDwukierunkowa 8

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

Projekt_1/Element.cpp 11
Projekt_1/Fabryka.cpp 11
Projekt_1/Iterator.cpp 11
Projekt_1/ListaDwukierunkowa.cpp 12

Chapter 3

Class Documentation

3.1 Element Class Reference

Tworzy nowy Element listy.

Public Member Functions

- **Element** (int wartosc)

Public Attributes

- int **wartosc**
- Element * **poprzedni**
- Element * **nastepny**

3.1.1 Detailed Description

Tworzy nowy Element listy.

Parameters

wartosc	Wartoæ przechowywana w Elemencie.

The documentation for this class was generated from the following file:

- Projekt_1/Element.cpp

3.2 Fabryka Class Reference

Tworzy nowy Element za pomoc[1] operatora new.

Static Public Member Functions

- static Element * **utworz** (int wartosc)

3.2.1 Detailed Description

Tworzy nowy Element za pomoc¹ operatora new.

Parameters

wartosc	Wartoæ przechowywana w Elemencie.

Returns

Wskanik na utworzony Element.

The documentation for this class was generated from the following file:

- Projekt_1/Fabryka.cpp

3.3 Iterator Class Reference

Public Member Functions

- Iterator (Element *start=nullptr)

  Tworzy iterator wskazuj¹cy na podany Element.
- void ustaw (Element *wezel)

  Ustawia iterator na wskazanym Elemencie.
- bool nastepny (int &wartosc)

  Zwraca wartoæ bie¿¹cego elementu i przesuwa iterator do przodu.
- bool poprzedni (int &wartosc)

  Zwraca wartoæ bie¿¹cego elementu i przesuwa iterator do ty³u.

3.3.1 Constructor & Destructor Documentation

3.3.1.1 Iterator()

```
Iterator::Iterator (
            Element * start = nullptr)  [inline], [explicit]
```

Tworzy iterator wskazuj¹cy na podany Element.

Parameters

start	Pocz¹tkowy Element iteratora.

3.3.2 Member Function Documentation

3.3.2.1 nastepny()

```
bool Iterator::nastepny (
            int & wartosc)  [inline]
```

Zwraca wartoæ bie¿¹cego elementu i przesuwa iterator do przodu.

Parameters

wartosc	Zmienna przechowuj¹ca odczytan¹ wartoæ.

Returns

true, jeli odczytano element, w przeciwnym razie false.

3.3.2.2 poprzedni()

```
bool Iterator::poprzedni (
            int & wartosc)  [inline]
```

Zwraca wartoæ bie¿¹cego elementu i przesuwa iterator do ty^{3}u.

Parameters

wartosc	Zmienna przechowuj¹ca odczytan¹ wartoæ.

Returns

true, jeli odczytano element, w przeciwnym razie false.

3.3.2.3 ustaw()

```
void Iterator::ustaw (
            Element * wezel)  [inline]
```

Ustawia iterator na wskazanym Elemencie.

Parameters

wezel	Element, na który ma wskazywaæ iterator.

The documentation for this class was generated from the following file:

- Projekt_1/Iterator.cpp

3.4 ListaDwukierunkowa Class Reference

Public Member Functions

- **ListaDwukierunkowa** (const ListaDwukierunkowa &)=delete
- ListaDwukierunkowa & **operator=** (const ListaDwukierunkowa &)=delete
- void dodajNaPoczatek (int wartosc)

  Dodaje element na pocz¹tku listy.
- void dodajNaKoniec (int wartosc)

  Dodaje element na koñcu listy.
- void dodajPodIndeks (int indeks, int wartosc)

  Dodaje element pod wskazanym indeksem.
- void **usunZPoczatku** ()

  Usuwa pierwszy element listy.
- void **usunZKonca** ()

  Usuwa ostatni element listy.
- void usunPodIndeks (int indeks)

  Usuwa element o wskazanym indeksie.
- void **wyswietl** () const

  Wywietla elementy listy od pocz¹tku do koñca.
- void **wyswietlOdwrotnie** () const

  Wywietla elementy listy od koñca do pocz¹tku.
- void **wyswietlNastepny** ()

  Wywietla bie¿¹cy element iteratora i przesuwa go do przodu.
- void **wyswietlPoprzedni** ()

  Wywietla bie¿¹cy element iteratora i przesuwa go do ty³u.
- void **wyczysc** ()

  Usuwa wszystkie elementy listy i zwalnia pamiêæ.

3.4.1 Member Function Documentation

3.4.1.1 dodajNaKoniec()

```
void ListaDwukierunkowa::dodajNaKoniec (
            int wartosc)  [inline]
```

Dodaje element na koñcu listy.

Parameters

wartosc	Wartoæ nowego elementu.

3.4.1.2 dodajNaPoczatek()

```
void ListaDwukierunkowa::dodajNaPoczatek (
            int wartosc)  [inline]
```

Dodaje element na pocz¹tku listy.

Parameters

wartosc	Wartoæ nowego elementu.

3.4.1.3 dodajPodIndeks()

```
void ListaDwukierunkowa::dodajPodIndeks (
            int indeks,
            int wartosc)  [inline]
```

Dodaje element pod wskazanym indeksem.

Parameters

indeks	Indeks wstawienia elementu.
wartosc	Wartoæ nowego elementu.

3.4.1.4 usunPodIndeks()

```
void ListaDwukierunkowa::usunPodIndeks (
            int indeks)  [inline]
```

Usuwa element o wskazanym indeksie.

Parameters

indeks	Indeks elementu do usuniêcia.

The documentation for this class was generated from the following file:

- Projekt_1/ListaDwukierunkowa.cpp

Chapter 4

File Documentation

4.1 Element.cpp

```
00001 #pragma once
00006 //Klasa która przechowóje wartoci i jest wykorzystywana jako element listy
00007 class Element {
00008 public:
00009     int wartosc;
00010     Element* poprzedni;
00011     Element* nastepny;
00012 
00013     Element(int wartosc) : wartosc(wartosc), poprzedni(nullptr), nastepny(nullptr) {}
00014 };
```

4.2 Fabryka.cpp

```
00001 #pragma once
00002 
00003 #include "Element.cpp"
00009 //Klasa która tworzy obiekty Element metod¹ fabryki
00010 class Fabryka {
00011 public:
00012     static Element* utworz(int wartosc) {
00013         return new Element(wartosc);
00014     }
00015 };
```

4.3 Iterator.cpp

```
00001 #pragma once
00002 
00003 #include "Element.cpp"
00004 
00005 //Klasa która jest iteratorem
00006 class Iterator {
00007 private:
00008     Element* aktualny;
00009 
00010 public:
00015     explicit Iterator(Element* start = nullptr)
00016         : aktualny(start) {
00017     }
00018 
00023     void ustaw(Element* wezel) {
00024         aktualny = wezel;
00025     }
00026 
00032     bool nastepny(int& wartosc) {
00033         if (aktualny == nullptr) {
00034             return false;
00035         }
```

```
00036
00037         wartosc = aktualny->wartosc;
00038         aktualny = aktualny->nastepny;
00039         return true;
00040     }
00041
00047     bool poprzedni(int& wartosc) {
00048         if (aktualny == nullptr) {
00049             return false;
00050         }
00051
00052         wartosc = aktualny->wartosc;
00053         aktualny = aktualny->poprzedni;
00054         return true;
00055     }
00056 };
```

4.4 ListaDwukierunkowa.cpp

```
00001 #pragma once
00002
00003 #include <iostream>
00004 #include "Element.cpp"
00005 #include "Fabryka.cpp"
00006 #include "Iterator.cpp"
00007
00008 //G3ówna klasa Listy dwukierunkowej
00009 class ListaDwukierunkowa {
00010 private:
00011     Element* glowa;
00012     Element* ogon;
00013     int rozmiar;
00014     Iterator iterator;
00015 public:
00016     ListaDwukierunkowa()
00017         : glowa(nullptr), ogon(nullptr), rozmiar(0), iterator(nullptr) {
00018     }
00019
00020     ListaDwukierunkowa(const ListaDwukierunkowa&) = delete;
00021     ListaDwukierunkowa& operator=(const ListaDwukierunkowa&) = delete;
00022
00023     ~ListaDwukierunkowa() {
00024         wyczysc();
00025     }
00030     // Metoda ktora dodaje element na poczatek
00031     void dodajNaPoczatek(int wartosc) {
00032         Element* nowy = Fabryka::utworz(wartosc);
00033
00034         nowy->nastepny = glowa;
00035
00036         if (glowa != nullptr) {
00037             glowa->poprzedni = nowy;
00038         }
00039         else {
00040             ogon = nowy;
00041         }
00042
00043         glowa = nowy;
00044         ++rozmiar;
00045         iterator.ustaw(glowa);
00046     }
00051     // Metoda ktora dodaje element na koniec
00052     void dodajNaKoniec(int wartosc) {
00053         Element* nowy = Fabryka::utworz(wartosc);
00054
00055         nowy->poprzedni = ogon;
00056
00057         if (ogon != nullptr) {
00058             ogon->nastepny = nowy;
00059         }
00060         else {
00061             glowa = nowy;
00062         }
00063
00064         ogon = nowy;
00065         ++rozmiar;
00066         iterator.ustaw(glowa);
00067     }
00073     // Metoda ktora dodaje element pod indeksem
00074     void dodajPodIndeks(int indeks, int wartosc) {
00075
00076         if (indeks == 0) {
00077             dodajNaPoczatek(wartosc);
```

```
00078             return;
00079         }
00080
00081         if (indeks == rozmiar) {
00082             dodajNaKoniec(wartosc);
00083             return;
00084         }
00085
00086         Element* obecny = glowa;
00087
00088         for (int i = 0; i < indeks; ++i) {
00089             obecny = obecny->nastepny;
00090         }
00091
00092         Element* nowy = Fabryka::utworz(wartosc);
00093
00094         nowy->poprzedni = obecny->poprzedni;
00095         nowy->nastepny = obecny;
00096         obecny->poprzedni->nastepny = nowy;
00097         obecny->poprzedni = nowy;
00098
00099         ++rozmiar;
00100         iterator.ustaw(glowa);
00101     }
00105     // Metoda ktora usuwa element z poczatku
00106     void usunZPoczatku() {
00107
00108         Element* usuwany = glowa;
00109         glowa = glowa->nastepny;
00110
00111         if (glowa != nullptr) {
00112             glowa->poprzedni = nullptr;
00113         }
00114         else {
00115             ogon = nullptr;
00116         }
00117
00118         delete usuwany;
00119         --rozmiar;
00120         iterator.ustaw(glowa);
00121     }
00125     // Metoda ktora usuwa element z konca
00126     void usunZKonca() {
00127
00128         Element* usuwany = ogon;
00129         ogon = ogon->poprzedni;
00130
00131         if (ogon != nullptr) {
00132             ogon->nastepny = nullptr;
00133         }
00134         else {
00135             glowa = nullptr;
00136         }
00137
00138         delete usuwany;
00139         --rozmiar;
00140         iterator.ustaw(glowa);
00141     }
00146     // Metoda ktora usuwa element pod indeksem
00147     void usunPodIndeks(int indeks) {
00148
00149         if (indeks == 0) {
00150             usunZPoczatku();
00151             return;
00152         }
00153
00154         if (indeks == rozmiar - 1) {
00155             usunZKonca();
00156             return;
00157         }
00158
00159         Element* usuwany = glowa;
00160
00161         for (int i = 0; i < indeks; ++i) {
00162             usuwany = usuwany->nastepny;
00163         }
00164
00165         usuwany->poprzedni->nastepny = usuwany->nastepny;
00166         usuwany->nastepny->poprzedni = usuwany->poprzedni;
00167
00168         delete usuwany;
00169         --rozmiar;
00170         iterator.ustaw(glowa);
00171     }
00175     // Metoda ktora wyswietla liste
00176     void wyswietl() const {
00177         Element* obecny = glowa;
```

```
00178
00179         std::cout « "Lista: ";
00180
00181         while (obecny != nullptr) {
00182             std::cout « obecny->wartosc « " ";
00183             obecny = obecny->nastepny;
00184         }
00185
00186         std::cout « '\n';
00187     }
00191     // Metoda ktora wyswietla liste odwrotnie
00192     void wyswietlOdwrotnie() const {
00193         Element* obecny = ogon;
00194
00195         std::cout « "Lista odwrotnie: ";
00196
00197         while (obecny != nullptr) {
00198             std::cout « obecny->wartosc « " ";
00199             obecny = obecny->poprzedni;
00200         }
00201
00202         std::cout « '\n';
00203     }
00207     // Metoda ktora wyswietla nastepny element
00208     void wyswietlNastepny() {
00209         int wartosc;
00210
00211         if (iterator.nastepny(wartosc)) {
00212             std::cout « "Nastepny element iteratora: "
00213                 « wartosc « '\n';
00214         }
00215         else {
00216             std::cout « "Brak nastepnego elementu.\n";
00217         }
00218     }
00222     // Metoda ktora wyswietla poprzedni element
00223     void wyswietlPoprzedni() {
00224         int wartosc;
00225
00226         if (iterator.poprzedni(wartosc)) {
00227             std::cout « "Poprzedni element iteratora: "
00228                 « wartosc « '\n';
00229         }
00230         else {
00231             std::cout « "Brak poprzedniego elementu.\n";
00232         }
00233     }
00237     // Metoda ktora czysci liste
00238     void wyczysc() {
00239         Element* obecny = glowa;
00240
00241         while (obecny != nullptr) {
00242             Element* nastepny = obecny->nastepny;
00243             delete obecny;
00244             obecny = nastepny;
00245         }
00246
00247         glowa = nullptr;
00248         ogon = nullptr;
00249         rozmiar = 0;
00250         iterator.ustaw(nullptr);
00251     }
00252 };
```

Index

dodajNaKoniec
    ListaDwukierunkowa, 8
dodajNaPoczatek
    ListaDwukierunkowa, 8
dodajPodIndeks
    ListaDwukierunkowa, 9

Element, 5

Fabryka, 5

Iterator, 6
    Iterator, 6
    nastepny, 7
    poprzedni, 7
    ustaw, 7

ListaDwukierunkowa, 8
    dodajNaKoniec, 8
    dodajNaPoczatek, 8
    dodajPodIndeks, 9
    usunPodIndeks, 9

nastepny
    Iterator, 7

poprzedni
    Iterator, 7
Projekt_1/Element.cpp, 11
Projekt_1/Fabryka.cpp, 11
Projekt_1/Iterator.cpp, 11
Projekt_1/ListaDwukierunkowa.cpp, 12

ustaw
    Iterator, 7
usunPodIndeks
    ListaDwukierunkowa, 9